WORDPRESS SETUP AND OPERATIONS · 29 OCTOBER 2026

# Restaurant Demo Setup guide

A generic restaurant website with an editable menu and table reservations. The local stack includes WordPress, MariaDB, and Mailpit for captured test email.

> **Local result:** the site was installed from a fresh Docker volume and the reservation flow passed in Chrome and the API smoke test. The existing site is available at `http://127.0.0.1:18087/`.

## 1. Requirements

- Docker Desktop with Docker Compose.
- PowerShell 7 or newer for `setup.ps1`.
- Node.js for the reservation smoke test; Python and Chrome for the optional Chrome MCP check.
- Free local ports 18087 and 18088, or two other free ports.

## 2. First local installation

Open PowerShell in the project directory. Check ports, set a unique admin password for this shell session, then run setup:

```
Get-NetTCPConnection -LocalPort 18087,18088 -State Listen -ErrorAction SilentlyContinue
$env:WP_ADMIN_PASSWORD = 'choose-a-long-unique-password'
.\setup.ps1
Remove-Item Env:WP_ADMIN_PASSWORD
```

On the first run, `setup.ps1` creates a private `.env` with random database passwords. It starts only this Compose project, installs WordPress, activates the theme, sets Post name permalinks and the Manila timezone, and creates the four pages and seven menu dishes. Running it again preserves existing bookings and the admin password.

Open `http://127.0.0.1:18087/`. On a fresh installation, sign in at `/wp-admin/` as `site_admin` with the password you chose. An existing installation keeps its original admin account. Open the captured email inbox at `http://127.0.0.1:18088/`.

If either default port is occupied, run `.\setup.ps1 -SitePort 18089 -MailpitPort 18090` using free ports. On a fresh install you can also supply `-AdminUser`, `-AdminEmail`, and `-SiteUrl`.

# 3. Edit and operate the site



| Task | Where |
| --- | --- |
| Edit dishes, prices, categories, and images | WordPress Admin → Menu dishes |
| View bookings and guest details | WordPress Admin → Reservations |
| Release a booking slot | Move that reservation to Trash |
| Review test emails | Mailpit on local port 18088 |

Guest bookings accept parties of 1–8, dates up to 60 days ahead, and at least two hours' notice. A booking is complete when its reference appears on the confirmation page. The current demo cap is ten parties per 90-minute slot.

# 4. Verify the installation

```
docker compose ps
node qa/reservation-smoke.mjs
python qa/chrome-mcp-check.py
```

The smoke test checks an invalid date, a valid booking, confirmation access, both guest and admin emails, and cleanup. The Chrome MCP test checks the home page, menu filter, mobile overflow, and a browser booking, then removes its test booking. Both tests need the local site running on the default ports.

To stop only this project, run `docker compose down`. Its database and WordPress volumes remain. Do not use `down -v` unless you intentionally want to erase its bookings and content.

# 5. Files and configuration

| Path | Purpose |
| --- | --- |
| `theme/` | Deployable WordPress theme; mounted read-only in Docker |
| `compose.yaml` | Local WordPress, database, Mailpit, and WP-CLI services |
| `.env` | Private local database credentials and ports; excluded from Git |
| `setup.ps1` | Repeatable WordPress setup |
| `qa/` | Automated checks and review screenshots |

Changing `DB_PASSWORD` in `.env` after a database already exists does not rotate its database user. Preserve the original value or perform an actual database password rotation. Keep `.env`, database backups, and guest records private.

# 6. VPS deployment verdict

Restaurant Demo · Local setup and deployment assessment

> **Not ready to expose as a real restaurant reservation service.** The Docker setup is a tested local environment. It binds WordPress to localhost, sends email into Mailpit, and uses initial business details. A public VPS needs the work below before accepting real guest data.

1. Put the site behind a reverse proxy with a real domain and valid HTTPS certificate. Set WordPress `home` and `siteurl` to the HTTPS domain; verify login and booking redirects.
2. Replace Mailpit with authenticated SMTP or a transactional email provider. Send and receive test messages to real addresses; remove the local mail capture service from production.
3. Replace sample address, opening hours, dishes, prices, photos, and contact instructions. Add a privacy notice and a working cancellation/contact path.
4. Define real table capacity and prevent concurrent overbooking. The current ten-party query is a demo limit and can race under simultaneous submissions.
5. Add abuse protection for the public reservation form, keep WordPress/themes/plugins patched, and use strong unique admin and database credentials.
6. Back up the database and `wp-content` off the VPS, test restoration, and monitor service health and email failures.

For a *public staging site with test data only*, use HTTPS and a reverse proxy, disable real reservations, and clearly label initial content. Do not publish the local `.env` or test admin credentials.

## References

WP-CLI install command · Official WordPress Docker image · Docker Compose environment variables · WordPress backup guidance